



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Python para Ciencia de Datos

Tema 7 Clases

Máster F.P. En IA & Big Data ANALYTICS
Curso 2026-2027

Resultados de aprendizaje

- Describir los conceptos de **objeto** y **método** en Python y saber determinar si dos referencias (variables) son en realidad el mismo objeto
- Explicar el concepto de **clase** en Python y su relación con los objetos o **instancias** de dicha clase
- Ser capaz de definir **nuevas clases** en Python, incorporando los atributos y métodos adecuados en función del problema a resolver, e instanciar y utilizar correctamente los objetos de dichas clases
- Ser capaz de crear **clases derivadas** de otras clases, utilizando el mecanismo de la **herencia** de Python
- **Utilizar** adecuadamente las **sentencias de control** de flujo en el contexto de programas sencillos en Python, incluyendo selección, bucles y gestión de excepciones
- Saber utilizar el mecanismo de **sobrescritura** de métodos de Python para adecuar el funcionamiento de clases derivadas, incluyendo el uso de ciertos operadores y funciones predefinidas de Python

Índice



- 1. Objetos**
- 2. Clases**
- 3. Herencia**
- 4. Sobrescritura**

Índice



1. Objetos

2. Clases

3. Herencia

4. Sobrescritura

Objetos

Concepto

- Python considera un **objeto** a cualquier **dato** en **memoria**
- Todo objeto tiene tres **características**:
 - Una **identidad**: es el valor que devuelve la función integrada `id()` aplicada a ese objeto (puede entenderse como su ubicación en memoria). Una vez creado, un objeto nunca cambia su identidad
 - Un **tipo**: es el valor que devuelve la función integrada `type()` aplicada sobre ese objeto. El tipo establece qué posibles valores puede tener el objeto y qué operaciones pueden aplicarse sobre él
 - Un **valor**: lo recibe cuando se crea el objeto, y dicho valor puede modificarse posteriormente si el tipo de objeto es mutable

Objetos

Función `id()` y operador `is/is not`

- Un mismo objeto puede tener simultáneamente varias **referencias**, es decir, distintas variables que referencian a dicho objeto
- Para saber si dos variables referencian o no el **mismo objeto** en memoria, podemos usar `id()` y los operadores `is/is not`

```
>>> x = 10
>>> y = 10
>>> print(id(x), id(y))
4377537928 4377537928
>>> y is x
True
>>> y = y + 1
>>> print(id(x), id(y))
4377537928 4377537960
>>> y is x
False
```

← `x` e `y` se crean de manera independiente, pero como tienen el mismo valor, realmente son referencias al mismo objeto

← Cuando modificamos el valor de `y` se crea un **nuevo** objeto con el valor modificado, y entonces ambas variables ya no apuntan al mismo objeto

Objetos

Función `id()` y operador `is/is not`

- Otros ejemplos:

```
>>> x = 10
>>> y = 10.0
>>> x is y
False
```

← **x** e **y** son de **tipos** de datos **distintos**, y por tanto no referencian el mismo objeto

```
>>> lista = [10, 20]
>>> lista2 = lista
>>> lista is lista2
True
>>> lista2.append(30)
>>> print(lista)
[10, 20, 30]
```

← Cuando **copiamos** una variable en otra, estamos copiando una **referencia** al objeto. Si el objeto es mutable, las modificaciones se realizan sobre el objeto en sí, y por tanto son visibles desde **cualquier** referencia que tenga

Objetos

Métodos

- Un **método** es una función definida para un cierto **tipo de datos** (o *clase*, como veremos más adelante) y que puede invocarse para cualquier **objeto** concreto de dicho tipo
- La **sintaxis** para invocar un método utiliza la “**notación punto**”, típica de la programación orientada a objetos:

```
objeto.método()
```

- Ejemplos:

```
>>> "Hello world!".split()  
['Hello', 'world!']  
>>> {1:'a', 2:'b'}.keys()  
dict_keys([1, 2])
```

```
>>> lista = [10]  
>>> lista.extend([20, 30])  
>>> print(lista)  
[10, 20, 30]
```

Objetos

Ventajas de los objetos

- La **programación** basada en objetos, u **orientada a objetos**, tiene múltiples ventajas:
 - **Organización** del código en función de abstracciones (**clases**) que reflejan los objetos del mundo real
 - **Encapsulación**: la definición de los objetos (clases) incluye su estado interno y sus posibles operaciones, obligando a definir interfaces de acceso y mejorando la reutilización de código
 - Características como la **herencia** y el **polimorfismo** (sobrecarga y sobrescritura) simplifican programación de sistemas complejos
 - Se mejora la **modularidad**, la **escalabilidad** y el **mantenimiento** del software, especialmente en programas grandes

Índice



1. Objetos

2. Clases

3. Herencia

4. Sobrescritura

Clases

Concepto

- Lo que hasta ahora definíamos como un tipo de dato en Python, en realidad es una **clase**
- Por tanto, una **clase** es una abstracción que permite definir un posible conjunto de valores y sus operaciones de acceso
- Todos los tipos de datos que hemos visto son en realidad **clases**, y podemos comprobarlo mediante la función `type()`:

```
>>> x = 10
>>> type(x)
<class 'int'>

>>> s = 'Hola'
>>> type(s)
<class 'str'>
```

```
>>> t = (10, 20, 30)
>>> type(t)
<class 'tuple'>

>>> type({1, 2})
<class 'set'>
```

Es indistinto aplicar `type()` a una variable o a un valor (objeto) literal

Clases

Clase vs objeto

- Es importante aclarar la diferencia entre una **clase** y un **objeto** (o **instancia**) de dicha clase
- Como hemos visto, una **clase** en Python se asocia al concepto de tipo de dato, y supone la **definición** (abstracta) de un conjunto de valores que comparten ciertas características y funcionalidades
- Desde esa perspectiva, un **objeto** o **instancia** es un dato que pertenece a una determinada clase, lo que establece sus posibles valores, sus métodos de acceso y su comportamiento interno

NOTA: En realidad, las clases en Python también son objetos, pero este concepto es complejo y queda fuera del contexto del curso

Clases

Creación de una nueva clase

- Podemos **crear** clases en función de las necesidades del programa
- La **sintaxis básica** para ello es:

```
class MiClase:  
    instrucción1  
    ...  
    instrucciónN
```

La convención es **nombrar**
las clases con las primeras
letras en **mayúscula**



donde:

- La palabra reservada **class** indica el inicio de la definición
- El bloque de instrucciones se ejecuta la primera vez que se "usa" la clase (normalmente al instanciar un objeto de dicha clase)

Clases

Creación de una nueva clase

- Las **instrucciones** habituales en una clase suelen ser de dos tipos:
 - Asignaciones a **variables locales** de la clase
 - Definición de **funciones**, que serán los **métodos** de acceso a los objetos de la clase
- Por ejemplo:

```
class MiClase:  
    i = 10
```

```
    def f(self):  
        return "Hello world!"
```

← i es una **variable** local de MiClase

← f es una **función** local de MiClase

Clases

Operaciones sobre una clase

- Una clase admite 2 tipos de **operaciones**: instanciación y referencia

1. Instanciación: es la creación de una instancia (es decir, un nuevo objeto) de dicha clase. Su sintaxis es:

```
class MiClase:  
    instrucción1  
    ...  
obj = Miclase()
```

Se **crea** un nuevo objeto de la clase MiClase y se **asigna** su referencia a la variable obj



- 2. Referencia** a los **atributos** (variables/funciones) de la clase
- Es posible acceder a los atributos de la clase (en el ejemplo anterior, `Miclase.i`) aunque poco habitual

Clases

Operaciones sobre objetos de una clase

- Una vez hemos creado (instanciado) un objeto de una clase, ¿qué **operaciones** podemos hacer con él?
 1. **Referencia** (acceso) a una **variable local** del objeto: los objetos pueden contener variables locales **propias** del objeto, que se crean de manera dinámica al asignarles un valor
 2. **Invocación** (llamada) a un **método** del objeto: cada función definida en la clase del objeto se convierte en un método disponible para cualquier objeto de la clase
- Ambas siguen la “notación punto”:

```
obj.v = 200  
obj.f()
```

Clases

Métodos: el término `self`

- Los **métodos** son funciones algo especiales, porque de manera implícita se asocian con el objeto para el que se invocan
- Para ello, deben **definirse** en la clase de la siguiente manera:

```
class MiClase:  
    i = 10  
  
    def f(self):  
        return "Hello world!"
```

```
obj = MiClase()  
c = obj.f()
```

El **primer parámetro** de la función representa el **objeto** desde el que se invocará el método. Se denomina siempre **self** (por convención)

La llamada a `f()` no incluye ningún parámetro, porque el **1º es implícito**, y corresponde al propio **objeto** (`obj`)

Clases

Inicialización de objetos: el método `__init__()`

- Muy habitualmente querremos que cada nuevo objeto de una clase tenga un **estado inicial** concreto
 - Ese estado inicial suele establecerse asignando valores a ciertas **variables locales** del objeto (a continuación)
- Se usa la función especial `__init__()` en la definición de la clase para establecer ese estado inicial de sus objetos:
 - Cuando **instanciamos** un nuevo objeto de una clase que tiene definida la función `__init__()`, esta función se **ejecuta** de manera **automática** durante la creación de ese objeto
 - Los **argumentos** especificados en la instanciación son recibidos por esta función

Clases

Inicialización de objetos: el método `__init__()`

- **Ejemplo:** Clase que representa un círculo de un color determinado

```
class Circulo:  
    PI = 3.1416  
  
    def __init__(self, radio, color):  
        self.radio = radio  
        self.color = color
```

```
c1 = Circulo(2.5, "rojo")  
c2 = Circulo(4.0, "azul")
```

Creamos **dos objetos** distintos de la clase (c1 y c2) e indicamos ciertos valores propios de cada uno en la llamada a la clase (el valor de su radio y su color)

Clases

Inicialización de objetos: el método `__init__()`

- **Ejemplo:** Clase que representa un círculo de un color determinado

```
class Circulo:  
    PI = 3.1416  
  
    def __init__(self, radio, color):  
        self.radio = radio  
        self.color = color
```

```
c1 = Circulo(2.5, "rojo")  
c2 = Circulo(4.0, "azul")
```

Durante la creación de `c1` y `c2` se llama **automáticamente** a `__init__`, pasándole los valores indicados en la instanciación

Se usa la **referencia** al mismo objeto (`self`) para asignar los valores a **variables locales**, que son propias de **cada** objeto

Clases

Variable de clase vs variable de objeto/instancia

- Al definir una clase podemos asignar valores a **variables de la clase**: son únicas para la clase, y compartidas por todas las instancias (objetos) de la clase (por ejemplo: PI)
- Cada objeto puede tener **variables de objeto** propias, no compartidas con ningún otro objeto de la clase

Pueden crearse al inicializar el objeto (en `__init__()`) o en cualquier otro momento (por ejemplo: `self.radio` o `self.color`)

```
class Circulo:  
    PI = 3.1416  
    def __init__(self, radio, color):  
        self.radio = radio  
        self.color = color  
  
c1 = Circulo(2.5, "rojo")  
c2 = Circulo(4.0, "azul")
```

Clases

Acceso a las variables de clases y objetos

- Normalmente, el acceso a las variables de clase u objeto se realiza desde dentro de la definición (código) de la clase
- Sin embargo, Python permite realizar ese acceso también desde **fuera** del código de la clase
- Incluso proporciona una **estructura de datos** estándar de tipo diccionario (**`__dict__`**) para consultar (o modificar) toda la información interna de sus clases u objetos
- Este tipo de accesos “externos” suele reservarse para inspeccionar los valores internos de una clase u objeto, con el objetivo de **depurar** un programa

Clases

Acceso a las variables de clases y objetos

- Ejemplo:

```
class Circulo:  
    PI = 3.1416  
  
    def __init__(self, radio, color):  
        self.radio = radio  
        self.color = color
```

```
c1 = Circulo(2.5, "rojo")  
c2 = Circulo(4.0, "azul")
```

```
print(Circulo.PI)  
print(c1.radio)  
c2.color = "negro"  
print(c2.__dict__)
```

Desde el **programa principal** podemos acceder a variables de la clase `Circulo` o de los objetos `c1` y `c2`. Observa que las variables pueden consultarse y modificarse, y que con `__dict__` tenemos acceso a todo el estado interno del objeto

```
3.1416  
2.5  
{'radio': 4.0, 'color': 'negro'}
```

Clases

Añadiendo funciones (métodos) a una clase

- En la definición de la clase, podemos añadir tantas funciones como queramos
- La **sintaxis** para ello es:

```
class MiClase:  
    ...  
    def nuevo_metodo(self [, p2, ...]):  
        instrucción1  
        ...
```

El primer parámetro, (**self**) es **obligatorio**. Opcionalmente puede tener más, tantos como requiera el método

Clases

Añadiendo funciones (métodos) a una clase

- **Ejemplo:** calcular el **área** del círculo

```
class Circulo:  
    PI = 3.1416  
  
    def __init__(self, radio, color):  
        self.radio = radio  
        self.color = color  
  
    def area(self):  
        return self.PI * (self.radio ** 2)  
  
c1 = Circulo(2.5, "rojo")  
c2 = Circulo(4.0, "azul")  
print(c2.area())
```

50.2656

Para calcular el área de un objeto círculo creamos un **método** que devuelve ese cálculo para dicho objeto

El método se invoca para un objeto (círculo) particular

Clases

Interfaz de acceso y encapsulación

- El conjunto de funciones de una clase (los métodos de sus objetos) se considera globalmente la **interfaz** de acceso a esos objetos
- Aunque como hemos visto en Python es posible acceder a las variables internas de una clase u objeto desde “fuera” de ellos, este uso está **desaconsejado**
- En programación orientada a objetos, la manipulación del estado interno (variables locales) de los objetos debe realizarse **solo** invocando sus **métodos**, y no desde el “programa principal”
- Esto permite la **encapsulación** de datos y operaciones, que es uno de sus principios básicos

Clases

Interfaz de acceso y encapsulación

- Si el programa que “usa” los objetos de una clase debe acceder a su **estado interno** (variables locales) para consultarlo o cambiarlo, debemos añadir a su interfaz **métodos** específicos para ello
- Ejemplo: consultar/modificar el radio de un círculo:

```
class Circulo:  
    ...  
  
    def get_radio(self):  
        return self.radio  
  
    def set_radio(self, nuevo_radio):  
        if nuevo_radio > 0:  
            self.radio = nuevo_radio
```

El acceso a las variables a través de la interfaz permite establecer **controles** sobre los valores que resultan admisibles para dichas variables (por ejemplo, radios no negativos)

Clases

Ejemplo completo

```
class Circulo:
    PI = 3.1416

    def __init__(self, radio, color):
        self.radio = radio
        self.color = color

    def area(self):
        return self.PI * (self.radio ** 2)

    def circunferencia(self):
        return self.PI * self.radio * 2

    def get_radio(self):
        return self.radio

    def get_color(self):
        return self.color

    def set_radio(self, nuevo_radio):
        if nuevo_radio > 0:
            self.radio = nuevo_radio

    def set_color(self, nuevo_color):
        self.color = nuevo_color
```

Índice

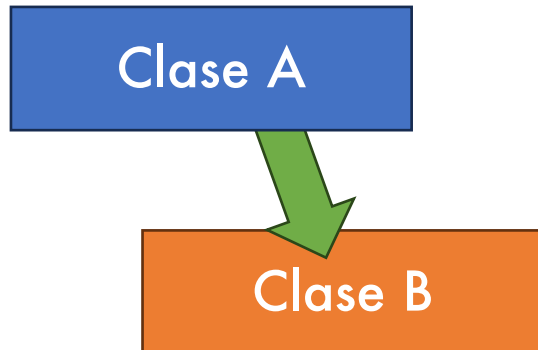


- 1. Objetos**
- 2. Clases**
- 3. Herencia**
- 4. Sobrescritura**

Herencia

Concepto

- La **herencia** es un concepto de programación orientada a objetos, que implica la posibilidad de crear **clases derivadas** de otras clases
- Cuando creamos una clase B derivada de otra A, la primera **hereda** todas las propiedades (**atributos** y **métodos**) de la segunda:



La **clase A** se denomina clase base, clase padre o superclase

La **clase B** se denomina clase derivada, clase hija o subclase

- De una clase derivada pueden, a su vez, derivarse otras

Herencia

Sintaxis

- Para crear una clase que herede de otra, la sintaxis es la siguiente:

```
class MiClase:  
    ...  
  
class ClaseDerivada(MiClase):  
    instrucción1  
    ...  
    instrucciónN
```

- Para definir ClaseDerivada como subclase de MiClase es necesario que esta última sea “accesible” (si está definida en otro módulo, debemos especificarla como <módulo>.MiClase)

Herencia

Aspectos importantes

- En la subclase podemos **añadir** nuevos métodos, pero también **redefinir métodos** de la superclase (volver a codificar un método con el mismo nombre que el de la superclase)
 - Esta redefinición se denomina **sobreescritura** (*overriding*)
- En la codificación de esos métodos, tenemos **acceso a** las variables de clase de la superclase y podemos invocar sus métodos
 - Para ello podemos usar la función **super()**
- Al instanciar **objetos** de la subclase, estos pueden invocar métodos de ambas clases (superclase y subclase). Si invocan un método redefinido en la subclase, se ejecutará este

Herencia

Ejemplo: polígono (superclase)

```
class Poligono:

    def __init__(self, *longitudes):
        self.n_lados = len(longitudes)
        if self.n_lados >= 3:
            self.lados = list(longitudes)
        else:
            raise ValueError # Error: faltan lados

    def perimetro(self):
        return sum(self.lados)

    def info(self):
        return f"Polígono de {self.n_lados} lados: {self.lados}"

p1 = Poligono(3, 4, 5)
p2 = Poligono(2, 2, 2, 2)
print("p1:", p1.info())
print("p2:", p2.info())
```

```
p1: Polígono de 3 lados: [3, 4, 5]
p2: Polígono de 4 lados: [2, 2, 2, 2]
```

Se **inicializa** un polígono cualquiera, con al menos 3 lados

El **perímetro** del polígono es la suma de sus lados

Se devuelve **información** genérica del polígono

Herencia

Ejemplo: triángulo (subclase)

```
class Triangulo(Poligono):  
    def __init__(self, *longitudes):  
        if len(longitudes) != 3:  
            raise ValueError  
        super().__init__(*longitudes)  
  
    def area(self):  
        a, b, c = self.lados  
        p = self.perimetro() / 2  
        return (p * (p - a) * (p - b) * (p - c)) ** 0.5  
  
    def info(self):  
        return f"Triángulo de lados: {self.lados}"  
  
t1 = Triangulo(3, 4, 5)  
print("t1:", t1.info())  
print("Perímetro de t1:", t1.perimetro())  
print("Área de t1:", t1.area())
```

Se crea la clase
Triangulo como
subclase de Poligono

Triangulo tiene una
inicialización específica y
luego la complementa
invocando la de Poligono
(referenciado mediante la
función **super()**)

Eso inicializa las variables
`self.n_lados` y
`self.lados`, que se
crean para el objeto
triángulo

Herencia

Ejemplo: triángulo (subclase)

```
class Triangulo(Poligono):

    def __init__(self, *longitudes):
        if len(longitudes) != 3:
            raise ValueError
        super().__init__(*longitudes)

    def area(self):
        a, b, c = self.lados
        p = self.perimetro() / 2
        return (p * (p - a) * (p - b) * (p - c)) ** 0.5

    def info(self):
        return f"Triángulo de lados: {self.lados}"

t1 = Triangulo(3, 4, 5)
print("t1:", t1.info())
print("Perímetro de t1:", t1.perimetro())
print("Área de t1:", t1.area())
```

Se añade una fórmula para calcular el **área** del triángulo partir de sus lados (fórmula específica de los triángulos)

Además, se **redefine** el método `info()` en la subclase

Herencia

Ejemplo: triángulo (subclase)

```
class Triangulo(Poligono):  
  
    def __init__(self, *longitudes):  
        if len(longitudes) != 3:  
            raise ValueError  
        super().__init__(*longitudes)  
  
    def area(self):  
        a, b, c = self.lados  
        p = self.perimetro() / 2  
        return (p * (p - a) * (p - b) * (p - c)) ** 0.5  
  
    def info(self):  
        return f"Triángulo de lados: {self.lados}"
```

```
t1 = Triangulo(3, 4, 5)  
print("t1:", t1.info())  
print("Perímetro de t1:", t1.perimetro())  
print("Área de t1:", t1.area())
```

```
t1: Triángulo de lados: [3, 4, 5]  
Perímetro de t1: 12  
Área de t1: 6.0
```

Las instancias de la clase Triangulo tienen disponibles **todos** los métodos de **ambas** clases

Si el método existe en la **subclase** (info(), area()), se invoca este.

En caso contrario (perimetro()), se invoca el de la **superclase**

Índice



- 1. Objetos**
- 2. Clases**
- 3. Herencia**
- 4. Sobrescritura**

Sobrescritura

Polimorfismo: sobrecarga y sobrescritura

- La **sobrecarga** y la **sobrescritura** son dos formas de **polimorfismo**, que es otro concepto clave de la programación orientada a objetos
- La **sobrecarga** supone que en una clase puede definirse una misma función (método) con **parámetros distintos**, para ofrecer usos alternativos de dicho método
 - Python lo soporta de manera indirecta mediante el uso de parámetros arbitrarios y con valor por defecto
- La **sobrescritura** supone que una clase derivada **redefine** una función (método) para adecuar su uso a objetos de la subclase
 - Python hace un uso extensivo mediante los **métodos especiales**

Sobrescritura

Métodos especiales

- Python proporciona un gran conjunto de **métodos especiales**, disponibles para cualquier clase, y que permiten redefinir para dicha clase, entre otros, **operadores** y **funciones predefinidas**
- Los métodos especiales se distinguen por una sintaxis particular: **empiezan y terminan** mediante **dos subrayados**
- De hecho, ya hemos usado uno de ellos: **`__init__()`**
 - Se invoca automáticamente al instanciar un nuevo objeto de una clase cualquiera, si dicha clase lo implementa (redefine)
- Vamos a ver algunos ejemplos más, agrupados en categorías

Sobrescritura

Métodos especiales para funciones predefinidas

Función	Método especial	Funcionalidad
<code>print(x)</code>	<code>__str__(self)</code>	Devuelve una cadena para ser impresa con información sobre x
<code>len(x)</code>	<code>__len__(self)</code>	Devuelve el tamaño de x
<code>bool(x)</code>	<code>__bool__(self)</code>	Devuelve si x debe interpretarse como True o False

IMPORTANTE: Toda clase puede sobrescribir cualquiera de esos métodos, pero solo debe hacerlo **si conceptualmente tiene sentido**

Por ejemplo, `__len__(self)` devuelve el “tamaño” de un objeto, normalmente indicando la cantidad de elementos que contiene. Si estamos definiendo una clase para la que no tiene sentido hablar de “tamaño”, seguramente no deberíamos sobrescribir este método

Sobrescritura

Métodos especiales para operadores de comparación

Operador	Método especial	Funcionalidad
<code>x < y</code>	<code>__lt__(self, other)</code>	¿Es x menor que y?
<code>x <= y</code>	<code>__le__(self, other)</code>	¿Es x menor o igual a y?
<code>x >= y</code>	<code>__ge__(self, other)</code>	¿Es x mayor o igual a y?
<code>x > y</code>	<code>__gt__(self, other)</code>	¿Es x mayor que y?
<code>x == y</code>	<code>__eq__(self, other)</code>	¿Es x igual a y?
<code>x != y</code>	<code>__ne__(self, other)</code>	¿Es x distinto de y?

En los métodos en los que hay dos parámetros (`self` y `other`) se supone que ambos son de la misma clase. En la implementación del método, podemos comprobar si `other` lo es mediante la función `isinstance(other, clase)`

Sobrescritura

Métodos especiales para operadores aritméticos

Operador	Método especial	Funcionalidad
<code>x + y</code>	<code>__add__(self, other)</code>	Calcula la suma de x e y
<code>x - y</code>	<code>__sub__(self, other)</code>	Calcula la diferencia entre x e y
<code>x * y</code>	<code>__mul__(self, other)</code>	Calcula la multiplicación de x por y
<code>x ** y</code>	<code>__pow__(self, other)</code>	Calcula x elevado a y
<code>x / y</code>	<code>__truediv__(self, other)</code>	Calcula la división de x entre y
<code>x // y</code>	<code>__floordiv__(self, other)</code>	Calcula la división entera de x entre y
<code>x % y</code>	<code>__mod__(self, other)</code>	Calcula el resto de dividir x entre y
<code>+x</code>	<code>__pos__(self)</code>	Devuelve x
<code>-x</code>	<code>__neg__(self)</code>	Devuelve -x

Sobrescritura

Ejemplo (1): Vector2D

Inicio de la **definición** de Vector2D:
Inicializar el objeto, calcular su **magnitud** y método para **imprimir** sus valores

```
import math

class Vector2D:
    """Clase que representa un vector 2D con coordenadas cartesianas."""

    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def magnitud(self):
        """Devuelve la magnitud (longitud) del vector."""
        return math.sqrt(self.x**2 + self.y**2)

    def __str__(self):
        return f"({self.x}, {self.y})"

...
```

Sobrescritura

Ejemplo (2): Vector2D

Sobrecarga
(sobrescritura) de
operadores
aritméticos: **suma**,
resta, **multiplicación**
de vectores, y vector
opuesto (negativo)

```
...  
# --- Sobrecarga de operadores aritméticos ---  
def __add__(self, other):  
    """Suma de vectores."""  
    if isinstance(other, Vector2D):  
        return Vector2D(self.x + other.x, self.y + other.y)  
    return NotImplemented  
  
def __sub__(self, other):  
    """Resta de vectores."""  
    if isinstance(other, Vector2D):  
        return Vector2D(self.x - other.x, self.y - other.y)  
    return NotImplemented  
  
def __mul__(self, other):  
    """Producto escalar entre dos vectores."""  
    if isinstance(other, Vector2D):  
        return self.x * other.x + self.y * other.y  
    return NotImplemented  
  
def __neg__(self):  
    """Negación unaria: devuelve el vector opuesto."""  
    return Vector2D(-self.x, -self.y)  
...
```

Sobrescritura

Ejemplo (3): Vector2D

Sobrecarga
(sobrescritura) de
operadores de
comparación,
basándose en la
magnitud de los
vectores comparados

```
...  
# --- Sobrecarga de ops. de comparación (basados en magnitud)  
def __eq__(self, other):  
    if isinstance(other, Vector2D):  
        return math.isclose(self.magnitud(), other.magnitud())  
    return NotImplemented  
  
def __lt__(self, other):  
    if isinstance(other, Vector2D):  
        return self.magnitud() < other.magnitud()  
    return NotImplemented  
  
def __le__(self, other):  
    if isinstance(other, Vector2D):  
        return self.magnitud() <= other.magnitud()  
    return NotImplemented  
  
def __gt__(self, other):  
    if isinstance(other, Vector2D):  
        return self.magnitud() > other.magnitud()  
    return NotImplemented  
  
def __ge__(self, other):  
    if isinstance(other, Vector2D):  
        return self.magnitud() >= other.magnitud()  
    return NotImplemented
```

Sobrescritura

Ejemplo (4): Vector2D

Uso de la clase:

```
v1 = Vector2D(3, 4)
v2 = Vector2D(1, 2)
v3 = Vector2D(-3, -4)

print("Vector v1:", v1)
print("Magnitud de v1:", v1.magnitud())

print("\nSuma v1 + v2:", v1 + v2)
print("Resta v1 - v2:", v1 - v2)
print("Vector opuesto de v1:", -v1)

print("\nProducto escalar v1 * v2:", v1 * v2)

print("\nComparaciones basadas en magnitud:")
print("v1 == v3 ?", v1 == v3)
print("v1 > v2 ?", v1 > v2)

print("v2 < 2 ?", v2 < 2) # Error!!
```

Vector v1: (3, 4)
Magnitud de v1: 5.0

Suma v1 + v2: (4, 6)
Resta v1 - v2: (2, 2)
Vector opuesto de v1: (-3, -4)

Producto escalar v1 * v2: 11

Comparaciones basadas en magnitud:
v1 == v3 ? True
v1 > v2 ? True

TypeError: '<' not supported between instances of 'Vector2D' and 'int'